技術者向け構築資料

M5Stack
在庫管理システム

AtomS3 / StickS3 + Unit QRCode
Google Sheets / Apps Script 構築・検証手順



Wi-Fi → Apps Script → Google Sheets

ARCHITECTURE · PROTOCOL · SETUP · ACCEPTANCE TEST

© 2026 K Visualization Studio

読取りから在庫表までの処理

端末はコードを保持し、Apps Scriptが履歴と在庫表を更新します。

1

2次元バーコードを読む

Unit
QRCodeのTRIGを
押している間に読取り



2

端末内へ保存

event_idを付け、
未確認キューへ保持



AtomS3 / StickS3

3

HTTPSで送信

Wi-Fi経由で
Apps ScriptへPOST

Google Apps Script

4

在庫を更新

Scansへ記録し、
Inventoryを集計

Google Sheets

在庫履歴はScansに保存し、Inventoryはその履歴から再集計します。

© 2026 K Visualization Studio

対応端末・リーダーと事前準備

AtomS3またはStickS3をUnit QRCodeと組み合わせます。



AtomS3

128×128画面 / PORT.A G2・G1



StickS3

135×240画面 / PORT.A G9・G10



Unit QRCode

TRIGボタン / I2Cモード

事前準備

- 2.4GHz Wi-Fi
- Googleアカウント
- Google Sheets
- VS Code + PlatformIO
- Grove / USB-Cケーブル

同じソースをM5Unified / M5GFXで動かし、端末ごとのPORT.Aと画面寸法は実行時に取得します。

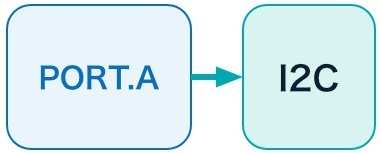
© 2026 K Visualization Studio

Unit QRCodeをI2Cモードで端末へ接続

AtomS3またはStickS3の電源を外してから、PORT.Aへ接続します。



いずれか1台



Groveケーブル



接続前の確認

1. 端末の電源を外す
2. Unit QRCodeの切替をI2Cへ
3. 端末のPORT.Aへ接続
4. 接続後にUSB給電

AtomS3: SDA=G2 / SCL=G1

StickS3: SDA=G9 / SCL=G10

ピン番号はソースへ固定しません

© 2026 K Visualization Studio

TRIGを押して読み取り、SAVEDを確認

操作はUnit QRCodeのTRIG。通信が早い場合はSTOREDから直接SAVEDへ進みます。

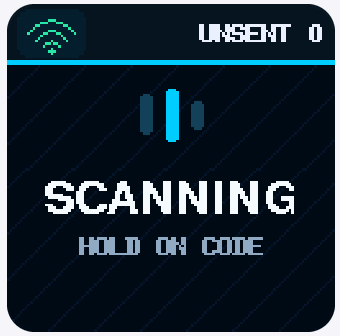
1



READY

次の商品を準備

2



TRIGを押す

押している間だけ読取り

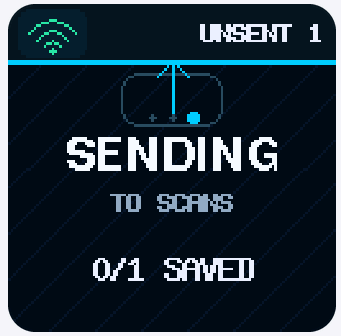
3



読取り音

音が鳴ったら離す

4



送信の表示例

通信状況により表示

5



SAVED

この1件をScansへ保存

SAVEDはこの1件をScansへ保存確認済み。通信中も次のTRIG操作ができます。
SENDING / SYNCINGは背景の再送中などに表示し、必ず全画面を順に表示するとは限りません。

© 2026 K Visualization Studio

画面表示の意味と次の操作

UNSENTは送信中も含む未確認件数。READYは次の受付可能、ALL SAVEDは全件完了です。



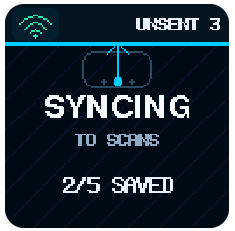
READY

次を読み取れます



STORED

端末内に保存済み



SYNCING

今回の対象中
確認済み件数



SAVED

この1件を
保存確認済み



ALL SAVED

未確認データなし



STORED

WAIT WIFI
復旧後に自動再送



NO CODE

もう一度読み取る



STORAGE FULL

未受付。空き後に
再読取

最大16件（送信中を含む）。満杯時の読取りは再スキャンが必要

Wi-Fi: 緑の弧 接続 / 橙の弧 接続中 / 灰の弧＋斜線 未接続

© 2026 K Visualization Studio

Inventoryシートで数量・状態・画像を確認

ブラウザを閉じていても、Webアプリが受信してデータを追加します。

A	B	C	D	E	F	G	H	I	J	K
コード	保有数	未使用	使用中	廃棄済み	製品名	代表画像	参照元	操作	管理	
A045-B	1	1	0	0	Unit NeoHEX		元画像		RESET INVENTORY Clear Scans + Inventory	
C126	4	4	0	0	M5Stack AtomS3R		元画像		FULL RESET Also clear ProductMaster	
C139	1	1	0	0	PaperS3		元画像			
U173	1	1	0	0	M5Stack Unit QRCode		元画像			

保有数 未使用＋使用中

状態 未使用・使用中・廃棄済み

商品情報 ProductMasterから表示

操作 状態変更を選択

© 2026 K Visualization Studio

3シートの役割とデータの流れ

履歴、商品情報、表示用集計を分離しています。

EVENT HISTORY

Scans

event_id / code / delta
received_at / 状態差分 /
action

登録と状態変更を追記
既存行は編集しない

PRODUCT METADATA

ProductMaster

code / name
imageURL / sourceURL

公式SKUから自動取得
失敗時はB〜D列を手動補完

DERIVED VIEW

Inventory

数量集計 + 完全一致参照
IMAGE / HYPERLINK

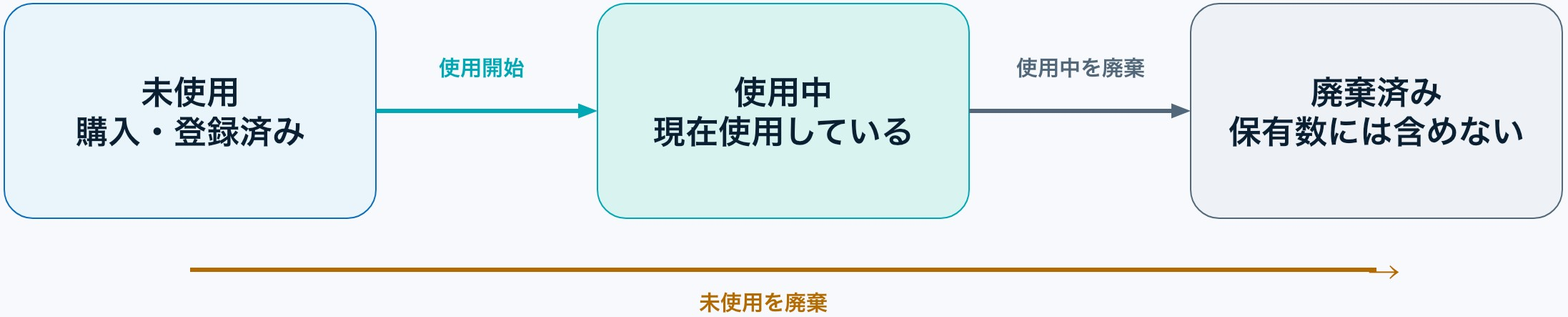
ScansとProductMasterから
再構成される表示

新規コードはScansへ必ず保存。商品検索に失敗しても在庫記録は残り、ProductMasterを後から補完できます。

© 2026 K Visualization Studio

在庫の状態はInventoryの操作欄から変更

数量セルやScansの履歴は直接編集しません。

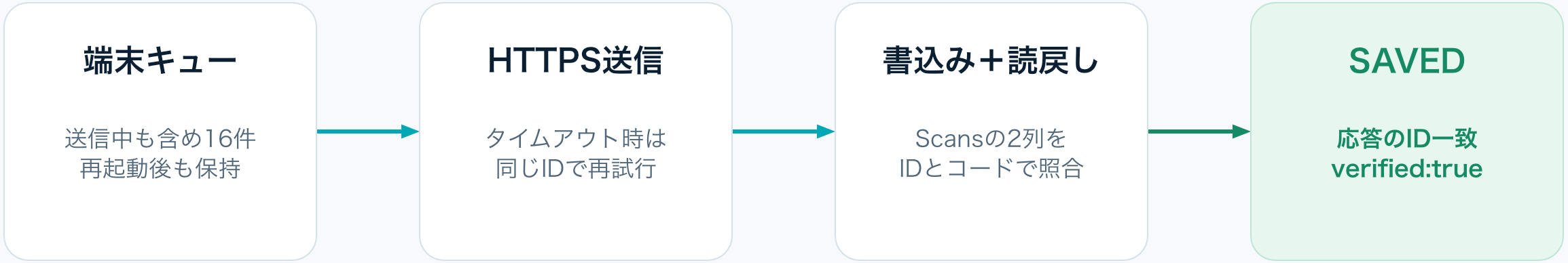


すべての状態変更はScansへ履歴として追加されます。
製品名・画像・参照元の修正はProductMasterで行います。

© 2026 K Visualization Studio

保存確認と二重登録の防止

同じevent_idを再送し、Google側で重複を判定します。



STOREDのとき

未確認データは削除されません。Wi-Fi復旧後、同じevent_idで自動再送されます。

確認後に永続キューを削除。失敗時は未送信のまま保持

© 2026 K Visualization Studio

端末とApps Scriptの受信プロトコル

HTTP 200だけでは成功とせず、JSON応答とeventIdを検証します。

```
POST /exec   application/json

{
  "version": 1,
  "key": "<DEVICE_KEY>",
  "eventId": "evt-...",
  "code": "C145"
}
```

eventId: 16〜80文字の英数字・ハイフン
code: 1〜512文字、制御文字なし / 1イベント=+1

保存確認応答

```
{"ok":true, "eventId":"evt-...",
 "duplicate":false, "verified":true}
```

ok:true + eventId一致 + verified:true + 端末内削除成功

ContentServiceのリダイレクト

302 / 303 → script.googleusercontent.com へGET
共有キーを含むPOSTは転送しない

再試行　BUSY / STORAGE_ERROR / timeout は同じeventId

UNAUTHORIZED / NOT_CONFIGURED / INVALID_REQUEST / SCHEMA_MISMATCH / EVENT_CONFLICT は設定・データを修正

© 2026 K Visualization Studio

Google SheetsからApps Scriptを開く

空の非公開スプレッドシートを作り、同梱ファイルを配置します。

1 スプレッドシートを作成

M5Stack_Inventory

ファイル名は任意

シートは非公開のまま
使用できます。

2 Apps Scriptを開く

拡張機能

Apps Script

ここから新しいタブを開く

3 2ファイルを配置

Code.gs

apps-script/Code.gs
の全内容を貼り付けて保存

appsscript.json

左の歯車でマニフェスト表示をオン
同梱内容へ置き換えて保存

appsscript.json：Asia/Tokyo / V8 / Sheets・外部取得権限 / 匿名Webアプリ設定

Google画面で「コード.gs」と表示されても問題ありません。

© 2026 K Visualization Studio

スクリプトプロパティを2件登録し、setupを実行

共有キーはコードへ直接書かず、Google側と端末側へ同じ値を設定します。

1 プロジェクトの設定

スクリプト プロパティ

SPREADSHEET_ID	URLの /d/ と /edit の間 ファイル名やシート名ではありません
DEVICE_KEY	32文字以上のランダムな共有キー 端末側にも同じ値を設定

例：`openssl rand -hex 32`

2 初回セットアップ

setup()を1回実行

1. 関数選択で setup を選ぶ
2. ［実行］を押す
3. Googleの権限確認を許可
4. 実行ログの「実行完了」を確認

作成されるシート　**Inventory / ProductMaster / Scans**

© 2026 K Visualization Studio

OSに合う方法で64文字のDEVICE_KEYを生成

32バイトの暗号学的乱数を16進数へ変換します。

macOS / Linux

OpenSSLを利用

```
openssl rand -hex 32
```

OpenSSLが利用できるターミナルで実行

出力例：a8f3…9c21（実際は64文字）

Windows 10 / 11

Windows PowerShell 5.1以上

```
$bytes = New-Object byte[] 32
$rng = [Security.Cryptography.RandomNumberGenerator]::Create()
$rng.GetBytes($bytes)
-join ($bytes | ForEach-Object { $_.ToString("x2") })
$rng.Dispose()
```

Windows標準機能のみ。OpenSSLの追加導入は不要

確認：0-9とa-fだけの64文字。Web上のランダム生成サイトは使用しない。

© 2026 K Visualization Studio

生成したDEVICE_KEYを2か所へ同じ値で設定

Google側と端末側が1文字でも違うと認証できません。

1 生成結果を確認

`64 hex characters`

前ページで生成した64文字が
このシステムのDEVICE_KEY

例：`a8f3…9c21`

→

2 Google側へ保存

プロジェクトの設定

スクリプト プロパティ

`DEVICE_KEY`

生成した64文字を貼り付け

→

3 端末側へ保存

platformio/include/secrets.h

`INVENTORY_DEVICE_KEY`

`"同じ64文字"`

確認方法

設定画面とsecrets.hで先頭4文字・末尾4文字を照合。UNAUTHORIZEDなら不一致、SAVEDなら認証と保存確認まで成功。

キー全体をチャット・README・動画へ貼らない

© 2026 K Visualization Studio

Apps ScriptをWebアプリとしてデプロイ

AtomS3またはStickS3が送信する /exec URLを発行します。

新しいデプロイ

種類	ウェブアプリ
次のユーザーとして実行	自分
アクセスできるユーザー	全員

```
https://script.google.com/macros/s/…/exec
```

発行後に確認

1. /exec で終わるURLを控える
2. secrets.h の ENDPOINTへ貼る
3. ブラウザで開き死活応答を確認
4. コード変更後は新バージョンへ更新

Sheets自体は非公開のままで利用可能

© 2026 K Visualization Studio

secrets.hにWi-Fi・送信先・証明書を設定

AtomS3とStickS3で同じ設定ファイルを使用します。

platformio/include/secrets.h

```
#define INVENTORY_WIFI_SSID "2.4GHz Wi-FiのSSID"
#define INVENTORY_WIFI_PASSWORD "Wi-Fiパスワード"
#define INVENTORY_ENDPOINT "https://script.google.com/macros/s/.../exec"
#define INVENTORY_DEVICE_KEY "Google側と同じDEVICE_KEY"

static const char INVENTORY_ROOT_CA[] = R"PEM(
-----BEGIN CERTIFICATE-----
... 証明書本文 ...
-----END CERTIFICATE-----
)PEM";




#ifndef INVENTORY_CAPTURE_ONLY
#define INVENTORY_CAPTURE_ONLY true  // 初回確認
#endif
```

既存の#ifndef〜#endifを残す。送信時は *-send-check を選択

SSID / PASSWORD

2.4GHz Wi-Fiの接続情報

ENDPOINT

手順12で発行した /exec URL

DEVICE_KEY

Google側と完全一致させる

ROOT_CA

script.google.com と
script.googleusercontent.com を検証

BEGIN/ENDを含むPEM全文を貼り付けます

secrets.hはGitへ登録・共有しない

© 2026 K Visualization Studio

ROOT_CAは現在の証明書チェーンを確認して設定

サーバー証明書を固定せず、信頼済みルートCAを照合します。

1 両ホストのチェーンを確認

```
openssl s_client -connect script.google.com:443 \
  -servername script.google.com -showcerts </dev/null

openssl s_client -connect script.googleusercontent.com:443 \
  -servername script.googleusercontent.com -showcerts </dev/null
```

2 Issuerをたどり、ルートCAを照合

OSの信頼済みCAストア、または pki.goog/repository/ で
名称・有効期限・SHA-256フィンガープリントを確認

1. ルートCAをPEMで取得
2. BEGIN / ENDを含めて連結
3. INVENTORY_ROOT_CAへ貼付
4. 実機ログでTLS成功を確認
5. 期限・Issuer変更時に更新

setInsecure() は使用しない

証明書チェーンは更新されるため、公開資料へ特定の古いルート名を固定しません。

© 2026 K Visualization Studio

接続中の端末に対応する環境を選ぶ

AtomS3とStickS3の書込みコマンドを取り違えないよう、どちらか一方だけを実行します。

リポジトリ直下から実行

```
cd platformio
```

AtomS3を接続した場合 — この1行だけ実行

```
pio run -e atoms3-send-check -t upload
```

StickS3を接続した場合 — この1行だけ実行

```
pio run -e sticks3-send-check -t upload
```

```
pio device monitor -b 115200
```

複数デバイス接続時は --upload-port で対象を明示

主要な固定バージョン

```
espressif32: AtomS3 6.9.0 / StickS3 6.12.0
M5Unified 0.2.22 / M5GFX 0.2.29 / M5UnitQRCode 1.0.1
```

実機で確認

1. Wi-Fi ON / READY
2. TRIGを押して1件読取
3. 未確認分はUNSENTで確認
4. SAVED / READY FOR NEXT
5. 全件完了：ALL SAVED / UNSENT 0

© 2026 K Visualization Studio

保存・再送・復旧をテスト用シートで確認

実在庫へ接続する前に、同一イベントが二重加算されないことを検証します。

自動確認

```
node --test tests/ingest.test.cjs
python3 tools/display-simulator/tests/run_tests.py

cd platformio
pio run -e atoms3 \
  -e atoms3-send-check \
  -e sticks3 \
  -e sticks3-send-check
```

追加試験：端末キュー・進捗・表示競合

実機シナリオ

1. 新規ID・新規コード → 数量1
2. 同じevent_idを再送 → 数量1のまま
3. 新しいID・同じコード → 数量2
4. Wi-Fi断・再起動後も未送信を保持
5. 再送中に追加。分母固定・FIFO順
6. 16件満杯の拒否と、空き後の再読取

合格条件：IDとコードの読戻し一致、保存件数・未送信数・全件完了がシナリオと一致

© 2026 K Visualization Studio

異常時の確認順と運用上の境界

履歴を直接削除せず、目的に応じて管理機能を使います。

よくある症状

UNAUTHORIZED	Google側と端末側のDEVICE_KEYを照合
UNSENTが減らない	Wi-Fi・ROOT_CA・/exec URL・実行履歴を確認
STORAGE FULL	空きができてから、未受付のコードを再読取
製品名や画像が空欄	ProductMasterのB〜D列と公式SKUページを確認
変更が反映されない	Webアプリを新しいバージョンへ更新
LOCAL STORAGE	端末内保存に失敗。端末・設定の状態を確認

データの扱い

RESET INVENTORY

ScansとInventoryを初期化
ProductMasterは保持

FULL RESET

3シートすべてを初期化
確認語「完全初期化」が必要

数量・数式・Scansの既存行は直接編集・削除しない

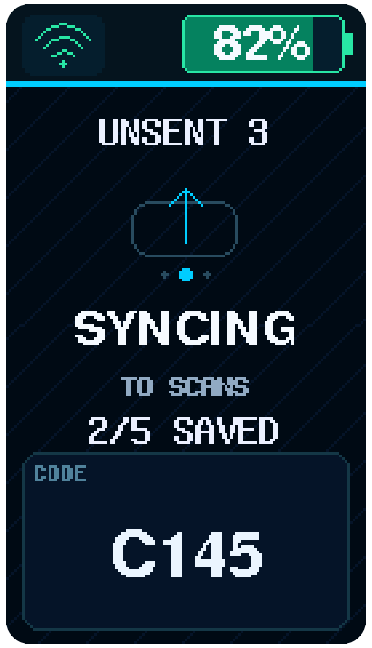
端末の状態表示、Scansの保存履歴、Inventoryの集計結果をそれぞれ確認します

制約：個人・少量データ向け / 履歴全体で重複確認 / Workspace匿名公開・クォータ要確認

© 2026 K Visualization Studio

StickS3の未送信数と再送進捗

Wi-Fiの右に残量%。UNSENTを独立表示し、コードと進捗を同時に確認できます。



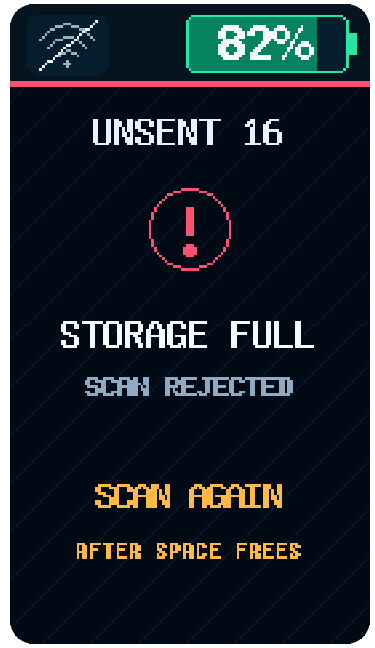
SYNCING

2/5 SAVEDは
今回の5件中2件
追加分は次の対象



ALL SAVED

端末の未確認0件
次のTRIGですぐ読取
READYへ戻る



STORAGE FULL

16件保持中
今回分は未受付
空き後に再スキャン

新しい読取りの表示を優先します。電池82%は例。取得できない場合は --% と表示します。
SAVEDは対象1件のScans保存確認。Inventoryの表示・製品名・画像更新とは別です。

© 2026 K Visualization Studio

StickS3の起動確認と切り分け

Unit起動時のブートモードから、電源を切り替えず通常の読取りへ戻すための機能です。

READYになるまで

① 初期化中もIRを消灯（GPIO46 LOW）
最初の給電後800msは通信せず待つ
正常0x21の設定を確認してREADYへ

② ブート0x54だけが応答した場合
復帰命令0x77を一度だけ送る
電源は切り替えない

③ 最大約2秒・10回で0x21だけを確認
版1〜254・手動1・TRIG 0/1まで
読戻しが正常ならREADYへ

起動・通信・復帰の段階を115200bpsで記録。
コード本文・Wi-Fi情報・キーは記録しません。

確認中と停止時の表示

STARTING / READER RESUME：復帰確認中
ERROR / RECOVERY HOLD：復帰未確認
READER STORAGE：復帰の記録保存に失敗
READER POWER / BUS / I2C：給電・通信
READER CONFIG：版・設定・TRIG読戻し

復帰命令の前に確認待ちを端末へ保存。
失敗や本体再起動後も0x54を再確認せず、
命令を再送しません。正常0x21の設定まで
確認できた場合だけ記録を解除します。
Aボタンで再試行せず、ログを確認します。
未送信イベントは保持。全消去しないでください。

ブート復帰は実機未検証です。Unit起動時の不調を対象とし、スキャン中の異常とは区別します。
電源原因の確定や再発防止の実証とは扱わず、READY・実読取り・Scans保存を別々に確認します。
TRIG未操作のCONTROL BYTESは読取りデータ検査の表示です。起動復帰・電源原因とは分けて確認します。

© 2026 K Visualization Studio